

UNIVERSIDADE FEDERAL DO PARANÁ

BRUNO YUUKI HAYASHI

SOMADORES DIGITAIS: do Half Adder ao Kogge-Stone

CURITIBA

2026

BRUNO YUUKI HAYASHI

SOMADORES DIGITAIS: do Half Adder ao Kogge-Stone

Trabalho apresentado como projeto de extensão NERD
como iniciativa à cultura de pesquisa

Orientador: Prof. Dr. Daniel Afonso Gonçalves de
Oliveira

CURITIBA
2025

"Projetamos uma nova era dourada para o campo da arquitetura de computadores na próxima década — assim como nos anos 1980, quando fizemos a pesquisa que nos levou ao prêmio."

— John L. Hennessy e David A. Patterson *A New Golden Age for Computer Architecture*, Communications of the ACM, v. 62, n. 2, p. 48, fev. 2019.

Resumo

Este trabalho mapeia os principais tipos de somadores digitais, partindo dos blocos mais básicos — o Half Adder e o Full Adder — e avançando por arquiteturas progressivamente mais sofisticadas: o Ripple Carry Adder, o Carry Lookahead Adder, o Carry Select Adder, o Carry Save Adder e o Kogge-Stone Adder. Para cada arquitetura, são discutidos o princípio de funcionamento, as vantagens e limitações, e a relação com as arquiteturas anteriores. Todas as implementações foram realizadas no simulador Logisim-evolution, permitindo verificar na prática os conceitos apresentados na teoria.

Palavras-chave: Somadores digitais. Aritmética binária. Carry. Arquitetura de Computadores.

SUMÁRIO

1. INTRODUÇÃO.....	6
2. REVISÃO TEÓRICA.....	7
2.1 Representação binária e operação de soma.....	7
2.2 Half Adder e Full Adder.....	8
2.3 Ripple Carry Adder (RCA).....	9
2.4 Carry Lookahead Adder (CLA).....	9
2.5 Carry Select Adder.....	10
2.6 Carry Save Adder (CSA).....	11
2.7 Kogge-Stone Adder.....	12
3. IMPLEMENTAÇÃO NO LOGISIM.....	14
4. CONSIDERAÇÕES FINAIS.....	15
5. REFERÊNCIAS.....	16

1. INTRODUÇÃO

A soma é a operação aritmética mais fundamental da computação digital. Quase tudo que um processador faz: subtrair, multiplicar, dividir, comparar... passa em algum momento por uma soma binária. Por isso, os somadores são componentes centrais de uma Unidade Lógica e Aritmética, a ULA, e entender como funcionam, é entender a base dos processadores.

No nível mais simples, somar dois números binários é um problema que aparenta ser simples, sendo resolvido com algumas portas lógicas. Porém, quando se deseja calcular muitos bits de forma rápida e eficiente, novos desafios surgem, e consequentemente, soluções também.

O principal ponto a se observar quando se prepara um somador, é o carry: o “vai um” que se propaga de bit em bit. Ao longo da história, muitas arquiteturas foram desenvolvidas propondo maneiras diversas de se resolver esse obstáculo; algumas priorizando simplicidade, outras implementação, velocidade, entre outros.

Essa pesquisa tem como objetivo mapear diferentes tipos de somadores digitais e apresentar a essência de cada um, discutindo também, sua implementação. O material servirá como base para a criação de um vídeo didático voltados ao conteúdo de arquitetura de computadores e design de circuitos integrados.

2. REVISÃO TEÓRICA

2.1 Representação binária e operação de soma

Diferente do sistema decimal que utiliza símbolos (0-9) e valores em potência de 10 para representar e organizar números, o sistema binário utiliza apenas dois: 0 e 1, e potência de 2. Cada posição de número binário, ou algarismo, se chama bit (binary digit), e seu conjunto representa um número, normalmente chamado de palavra binária (binary word).

Tabela 1 - comparação entre representação decimal e binária de números

Decimal	Binário
0	0
1	1
2	10
3	11
4	100
10	1010

A soma de dois bits, ocorre de forma semelhante ao sistema decimal. É feita entre dois “números” e, quando acontece “1+1”, como não existe a representação “2”, acontece o caso de *carry*, em que “soma 1 na próxima casa” (como se ocorresse 9+1). O carry é o que torna a soma multi bit mais complexa do que aparenta.

Tabela 2 - casos possíveis da soma entre dois bits

A	B	Soma	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

A soma multi bit, já deve receber 3 variáveis: A, B e um possível carry da soma anterior, funcionando da mesma forma que uma soma decimal comum, porém, com as limitações do sistema binário. Esse comportamento de receber 3 entradas, e ter como saída o valor da soma e o carry (0 para não carry e 1 para carry), é o que o somador completo (full adder) implementa.

Antes de chegar nos somadores em si, é importante ter dois conceitos esclarecidos, a geração e a propagação do carry. Um carry é gerado, independente do carry de entrada, quando é realizado a soma $A+B$, com $A=1$ e $B=1$; nesse caso, é produzido um carry de saída.

A propagação ocorre quando ou $A=1$ ou $B=1$, mas não ambos ao mesmo tempo; dessa forma, se há um carry de entrada, é gerado um carry de saída.

2.2 Half Adder e Full Adder

O meio-somador, ou half adder, é o circuito mais simples que pode realizar uma soma binária. Recebe dois bits de entrada (A e B) e produz dois bits de saída: soma (S) e o carry de saída (Cout). Sua tabela verdade é igual a tabela de soma entre dois bits, possível verificar em: “Tabela 2 - casos possíveis da soma entre dois bits”. Observando a tabela, é possível identificar duas funções lógicas, a saída de soma é idêntica a de uma porta XOR, e a saída de Cout é a de uma porta AND.

Sendo assim, o circuito é composto por apenas duas portas lógicas, uma AND e uma XOR. Apesar de ser simples e fácil de montar, sua limitação está nas somas em cadeia, ou multi bits, em que é necessário uma entrada para o carry de entrada (Cin). Por isso, ele só pode ser usado na coluna mais à direita de uma soma, em que não há carry de entrada. Expressões de S e Cout do Half Adder:

$$S = A \oplus B$$

$$Cout = A \cdot B$$

Por isso, existe o somador completo (Full Adder). O somador completo, estende o meio-somador, adicionando a terceira entrada Cin. Assim, ele consegue processar qualquer soma multi bit, incluindo as que recebem carry da “coluna” anterior.

Tabela 3 - Tabela verdade Full Adder

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Legenda: azul- entradas; verde- saídas

Com a tabela, é possível calcular as expressões para S e Cout. A soma S é uma expressão de XOR tripla, e Cout é uma expressão um pouco mais complexa, que pode ser reescrita utilizando geração (G) e propagação (P) vistos anteriormente. Expressões de saídas Full Adder:

$$S = A \oplus B \oplus Cin$$

$$\begin{aligned} \text{Cout} &= (A \cdot B) + (A \cdot \text{Cin}) + (B \cdot \text{Cin}) \\ &\text{ou} \\ \text{Cout} &= G + (P \cdot \text{Cin}) = (A \cdot B) + ((A \oplus B) \cdot \text{Cin}) \end{aligned}$$

2.3 Ripple Carry Adder (RCA)

O Ripple Carry Adder, conhecido também como somador de carry em cascata, é a arquitetura mais direta para realizar a soma de dois números binários com múltiplos bits. A construção consiste em encadear N Full Adders em série, de forma que o Cout de cada estágio, seja conectado diretamente ao Cin do estágio seguinte, como uma soma coluna por coluna. O bit menos significativo (Least Significant Bit, LSB) recebe $\text{Cin} = 0$, pois não há carry anterior.

A partir disso, os somadores processam seus bits (A, B) junto ao carry vindo do estado anterior, produzindo o bit de Soma e propagando o carry adiante. O bit mais significativo (Most Significant Bit, MSB) produz o Cout final, que indica overflow caso os operandos sejam tratados com números sem sinal.

Para um somador de 4 bits, a cadeia se organiza da seguinte forma:

$$\text{Cin}=0 \rightarrow \text{FA}_0(A_0, B_0) \rightarrow C_1 \rightarrow \text{FA}_1(A_1, B_1) \rightarrow C_2 \rightarrow \text{FA}_2(A_2, B_2) \rightarrow C_3 \rightarrow \text{FA}_3(A_3, B_3) \rightarrow \text{Cout}$$

A principal vantagem do RCA é a simplicidade, é regular, escalável e de fácil implementação, bastando replicar o Full Adder quantas vezes forem necessárias. Entretanto, a limitação está é o tempo de propagação do carry, que no pior caso, deve passar por todos os Full Adders em sequência. Considerando que um Full Adder, introduz um atraso de propagação, o atraso total do somador cresce linearmente com o número de bits.

Por conta disso, o RCA é progressivamente mais lento à medida que o número de bits aumenta, tornando-o inadequado para aplicações de palavras largas (16, 32, 64 bits...). Essa limitação é justamente a motivação para as próximas arquiteturas.

2.4 Carry Lookahead Adder (CLA)

O Carry Lookahead surge como resposta direta à limitação do RCA. Em vez de esperar que o carry se propague sequencialmente por cada estágio, o CLA calcula todos os carries paralelamente, utilizando expressões lógicas derivadas das entradas A e B. Para conseguir fazer isso, o CLA utiliza dos conceitos de geração e propagação vistos anteriormente.

$$\begin{aligned} G_i &= A_i \cdot B_i \quad (\text{carry é gerado neste estágio}) \\ P_i &= A_i \oplus B_i \quad (\text{carry é propagado neste estágio}) \end{aligned}$$

Com essas expressões, é possível descrever o carry de saída de qualquer estágio sem depender do carry calculado pelo anterior. O carry de entrada do estágio $i+1$ (próximo) é dado por:

$$C_{i+1} = G_i + (P_i \cdot C_i)$$

Expandindo a recorrência a partir de $C_0 = 0$, obtém-se as expressões de cada carry.

$$\begin{aligned}C_1 &= G_0 + P_0 \cdot C_0 \\C_2 &= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0 \\C_3 &= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0 \\C_4 &= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0 \\&\dots\end{aligned}$$

É possível observar que todos os carries dependem apenas das entradas A e B e do carry inicial que é 0. O atraso deixa de ser linear N e passa a ser aproximadamente uma constante.

A estrutura interna é composta por três elementos principais. Primeiro, uma camada de geração de G e P que calcula os sinais para cada bit em paralelo. Em segundo, um bloco lookahead, que implementa as expressões para calcular os carries. Terceiro, uma camada de somadores que com os carries, computa os bits de soma.

$$S_i = P_i \oplus C_i$$

O problema do CLA é o custo em hardware que cresce com a expansão do somador. A cada estágio, as expressões dos carries aumentam e sua complexidade também. Por isso, na prática, as CLAs são organizadas em blocos de 4 bits encadeados entre si.

2.5 Carry Select Adder

O Carry Select Adder tem uma forma diferente de contornar a espera pelo carry: ao invés de calcular antecipadamente, realiza os dois cálculos possíveis ao mesmo tempo e seleciona o resultado correto quando o carry chega.

A ideia é dividir o somador em blocos de bits. Para cada bloco, são postos dois RCAs em paralelo, um assumindo $C_{in} = 0$ e outro $C_{in} = 1$. Ambos calculam o resultado simultaneamente enquanto o bloco anterior ainda está propagando o carry. Quando o carry fica disponível, um MUX seleciona qual dos resultados é correto.

$$\begin{array}{l} \text{Bloco 0: } FA_0-FA_3 \rightarrow \text{calcula normalmente, produz } C_4 \\ \downarrow \\ \text{Bloco 1A (Cin=0): } FA_4-FA_7 \rightarrow S[7:4] \text{ e Cout assumindo carry=0 } \neg \\ \text{Bloco 1B (Cin=1): } FA_4-FA_7 \rightarrow S[7:4] \text{ e Cout assumindo carry=1 } \neg \rightarrow \text{MUX(sel=C}_4\text{)} \rightarrow \\ \text{resultado final} \end{array}$$

O primeiro bloco, não é duplicado pois $C_0 = 0$. A partir do segundo em diante, todos os blocos operam em paralelo, com seus RCAs internos. O atraso total do somador passa a ser o atraso do primeiro bloco somado ao atraso de um MUX, independente de quantos blocos intermediários existam. O atraso pode ser expresso por:

$$T_{\text{CarrySelect}} \approx T_{\text{bloco0}} + T_{\text{MUX}}$$

Em comparação com o CLA, o Carry Select Adder é mais simples de implementar, pois reutiliza diretamente a estrutura do Full Adder, sem a lógica do lookahead. Em contrapartida, o custo do bloco exige o dobro de Full Adders, além dos MUXs.

Uma variação, é a utilização de blocos de tamanhos crescentes ao longo do somador. Como os blocos iniciais recebem seus carries mais cedo, blocos maiores no final da cadeia permitem que o tempo de cálculo dos RCAs coincida com a chegada do carry, equilibrando o pipeline e otimizando a relação entre área e desempenho

2.6 Carry Save Adder (CSA)

O Carry Save Adder, diferente dos somadores anteriores que foram projetados para somar dois operandos de N bits, o CSA é otimizado para somar três ou mais operandos simultaneamente, sendo utilizado como componente interno de multiplicadores e acumuladores.

O ponto central do CSA é que o problema das somas encadeadas não está na soma, mas na propagação do carry. Se for realizar por exemplo, $A+B+C$, em primeira mão é comum pensar primeiramente $A+B$, esperando os carries e o resultado estarem completos, após isso, fazer o resultado $+ C$. O CSA elimina essa espera.

Em vez de propagar o carry imediatamente, o CSA “salva” ele. Cada Full Adder do bloco recebe três bits de entrada (A, B e C) e tem como saída o bit de soma S e o carry C, sem propagá-lo. O resultado não é o número pronto, mas sim, dois vetores: um vetor de somas e um de carries salvos, deslocado uma posição para a esquerda. A operação de um estágio CSA produz:

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{out_i} = (A_i \cdot B_i) + (A_i \cdot C_i) + (B_i \cdot C_i)$$

Essas são as expressões do Full Adder, a mudança é qque o Cout não é conectado ao próximo estágio, mas sim ao vetor de carries. O resultado final é obtido somando os dois vetores com um somador normal, RCA ou CLA por exemplo.

$$\begin{array}{r} A = 1\ 0\ 1\ 1 \\ B = 0\ 1\ 1\ 0 \\ C = 1\ 1\ 0\ 1 \\ \downarrow \text{CSA} \\ S = 0\ 0\ 0\ 0 \\ Cc = 1\ 1\ 1\ 0 \\ \downarrow \text{somador final (RCA ou CLA)} \\ \text{resultado final} \end{array}$$

O ganho de desempenho é notório quando o CSA é utilizado em árvore, para somar muitos operandos. Existem árvores em que, multiplos estágios CSA reduzem N operandos a apenas dois vetores intermediários em tempo $O(\log N)$, e apenas uma soma final com propagação de carry é necessária. Por isso, o CSA é fundamental em multiplicadores de alto desempenho.

O ponto negativo do CSA é que ele não gera um resultado imediato. Os dois vetores finais precisam de um somador final para produzir o número completo, em outras palavras, ele nunca opera de forma isolada, é sempre um componente dentro de uma estrutura maior.

2.7 Kogge-Stone Adder

O Kogge-Stone Adder pertence ao grupo de somadores paralelos e é uma das arquiteturas de melhor desempenho para somas binárias. Enquanto o CLA calcula os carries antecipadamente para um bloco fixo de bits, o Kogge-Stone generaliza essa ideia em uma árvore que calcula todos os carries do somador em paralelo, com o atraso crescendo logaritmicamente com o número de bits.

A base desse somador é definir um operador fixo ($\bullet$) que combina com dois pares (G, P) de estágios adjacentes em um único par agregado:

$$(G_alto, P_alto) \bullet (G_baixo, P_baixo) = (G_alto + P_alto \cdot G_baixo, P_alto \cdot P_baixo)$$

O par resultante representa um bloco que gera carry se: o estágio alto gera ou se o estágio alto propaga e o estágio baixo gera. O carry se propaga pelo bloco inteiro apenas se ambos estágios propagarem. O operador é associativo, o que permite organizá-lo livremente em árvore sem alterar o resultado.

O objetivo é calcular, para cada posição i o par prefixo acumulado desde o bit 0 até o i . Esse par, chamado $(G_{0:i}, P_{0:i})$, representa a geração e propagação do carry considerando todos os bits do LSB até i . Com ele, o carry de entrada do estágio $i+1$ é simplesmente:

$$C_{i+1} = G_{0:i} + P_{0:i} \cdot C_0$$

Como $C_0 = 0$, na prática $C_{i+1} = G_{0:i}$ para todos os estágios.

O Kogge-Stone organiza o cálculo desses pares acumulados em $\lg(N)$ níveis de profundidade. Em cada nível k , cada nó combina seu par local com o par do nó que está $2^{(k-1)}$ posições à esquerda. Para um somador de 8 bits, a árvore se desenvolve em 3 níveis:

Nível 0 (entradas): $G_0P_0 \ G_1P_1 \ G_2P_2 \ G_3P_3 \ G_4P_4 \ G_5P_5 \ G_6P_6 \ G_7P_7$

Nível 1 (passo=1): combina cada nó com o vizinho imediatamente à esquerda

Nível 2 (passo=2): combina cada nó com o nó 2 posições à esquerda

Nível 3 (passo=4): combina cada nó com o nó 4 posições à esquerda

Saída: $G_{0:0} \ G_{0:1} \ G_{0:2} \ G_{0:3} \ G_{0:4} \ G_{0:5} \ G_{0:6} \ G_{0:7}$

Ao final dos $\lg(N)$ níveis, cada posição i possui seu par prefixo acumulado completo, e todos os carries estão disponíveis simultaneamente. Os bits de soma são calculados da mesma forma que no CLA:

$$S_i = P_i \oplus C_i$$

O atraso do Kogge-Stone cresce com $O(\lg N)$, o que o torna significativamente mais rápido que o RCA ($O(N)$) para palavras largas, e comparável ao CLA, porém, com estrutura mais regular e sem complexidade de múltiplos níveis de blocos. Para um somador de 64 bits por exemplo, Kogge-Stone requer 6 níveis de lógica para calcular todos os carries.

O ponto negativo é o custo em área e interconexões. Cada nível da árvore possui N nós ativos, e as conexões entre nós de níveis distantes tornam o roteamento em hardware físico mais complexo. Para $N=64$, o número de operadores prefixo é $N \times \lg(N)/2 = 192$, contra os $N-1 = 63$ Full Adders de um RCA equivalente.

3. IMPLEMENTAÇÃO NO LOGISIM

O Logisim Evolution é uma ferramenta de simulação de circuitos digitais que permite construir e testar circuitos lógicos de forma visual e interativa. Nesta seção são apresentadas as implementações dos somadores estudados na revisão teórica, destacando as decisões de projeto e os pontos de atenção de cada circuito. Todos os somadores foram implementados para operandos de 4 bits, com exceção do Carry Select Adder, implementado com 8 bits para evidenciar a duplicação de blocos, e do Kogge-Stone Adder, implementado com 8 bits para que a estrutura em árvore fique visível.

Uma prática adotada em todas as implementações é o uso de subcircuitos. O Logisim permite encapsular um circuito em um componente reutilizável, o que torna os projetos mais organizados e reflete diretamente a modularidade presente nas arquiteturas estudadas. Todos os somadores foram construídos progressivamente, cada um reutilizando subcircuitos dos anteriores.

3.1 Half Adder e Full Adder

O Half Adder é o ponto de partida da implementação. Por ser composto por apenas uma porta XOR e uma porta AND, sua montagem é direta: as entradas A e B alimentam ambas as portas simultaneamente, com a saída da XOR correspondendo à soma S e a saída da AND ao carry Cout. Ao final, o circuito é encapsulado como subcircuito para ser reutilizado na construção do Full Adder.

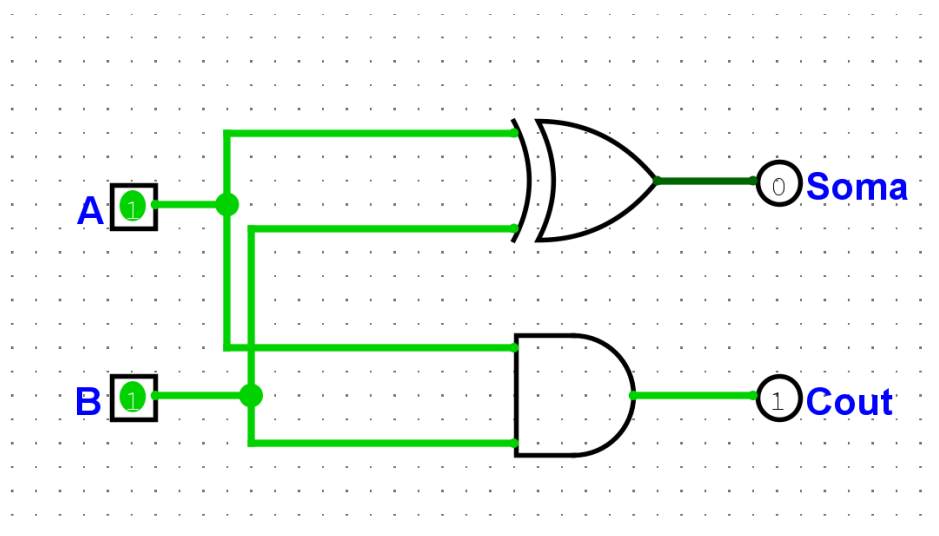


Imagem 1 - Circuito completo Half Adder

O Full Adder é construído compondo dois subcircuitos Half Adder com uma porta OR. O primeiro Half Adder recebe A e B e produz uma soma parcial e um carry parcial. O

segundo Half Adder recebe essa soma parcial e o Cin, produzindo a soma final S. A porta OR combina os dois carries parciais e produz o Cout final. Essa abordagem demonstra na prática o princípio de design modular o mesmo subcircuito reutilizado duas vezes dentro de um circuito maior.

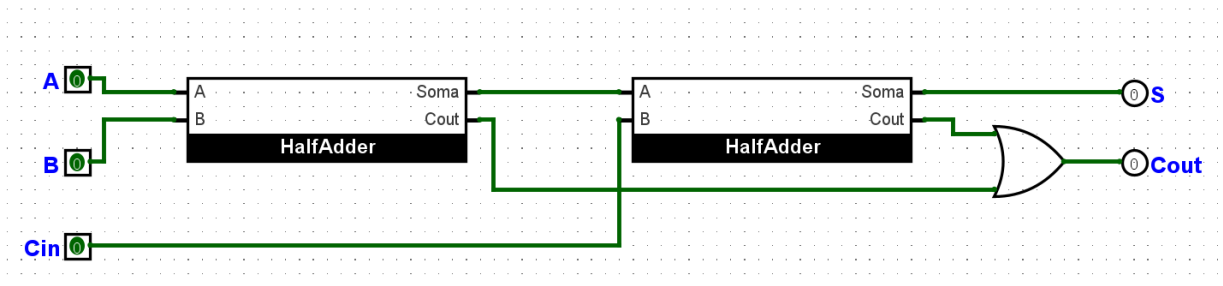


Imagem 2 - Circuito completo FullAdder

3.2 Ripple Carry Adder

O RCA de 4 bits é construído instanciando quatro subcircuitos Full Adder em sequência. O ponto central da implementação é o encadeamento do carry: o Cout de cada Full Adder é conectado ao Cin do seguinte, partindo de Cin fixo em zero no primeiro estágio. As entradas A e B são fornecidas como barramentos de 4 bits, separados por splitters em bits individuais para alimentar cada Full Adder. As saídas de soma são reunidas por um splitter inverso em um barramento de 4 bits.

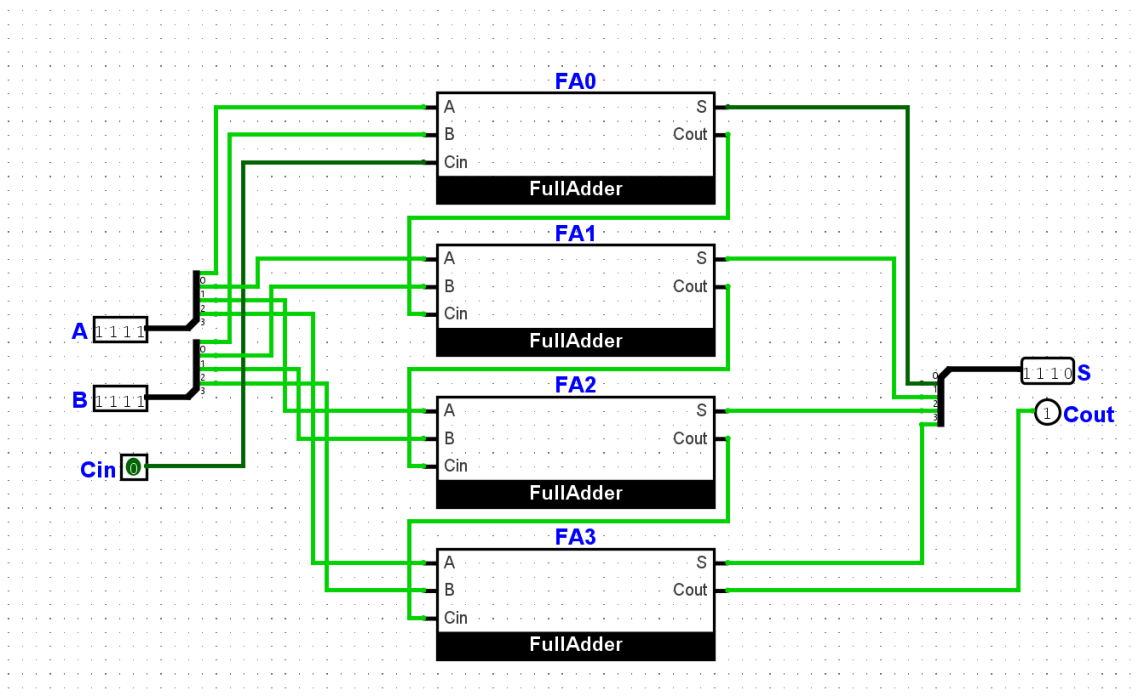


Imagem 3 - Circuito completo Ripple Carry Adder

3.3 Carry Lookahead Adder

O CLA de 4 bits é organizado em três camadas visualmente separadas no canvas. A primeira camada calcula os sinais G e P para cada bit com portas AND e XOR. A segunda camada é o bloco lookahead, que implementa as expressões de C1 a C4 usando portas AND e OR. A terceira camada calcula os bits de soma com portas XOR, combinando o P inicial de cada bit com o carry calculado pelo bloco lookahead.

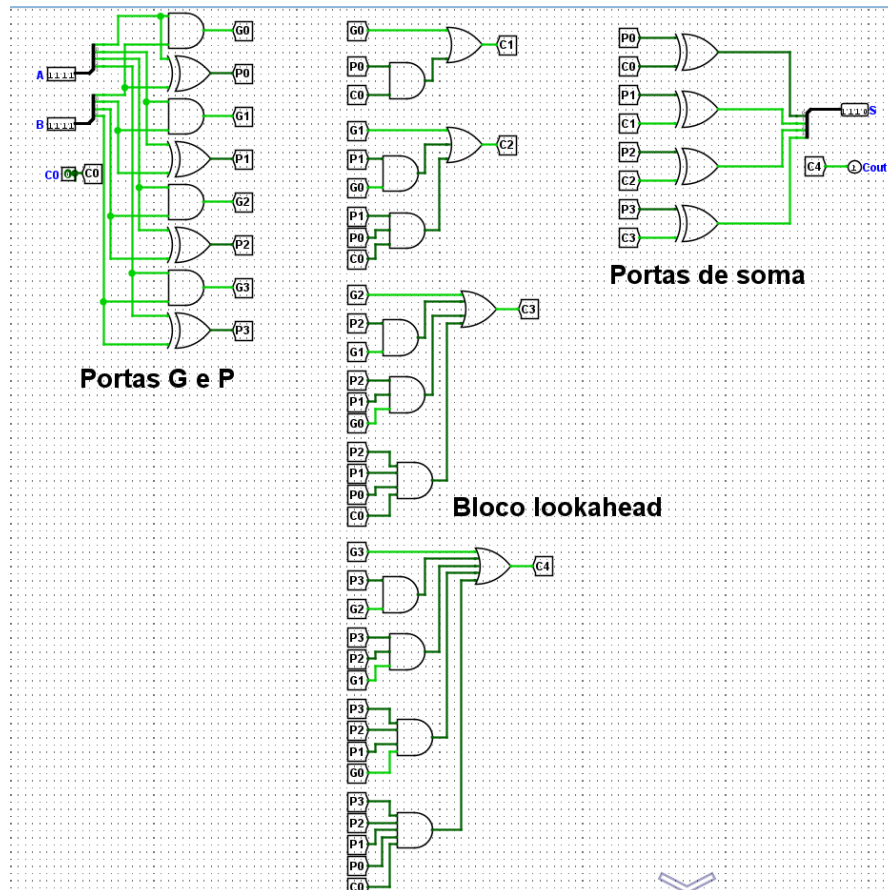


Imagem 4 - Circuito completo Carry Lookahead Adder

3.4 Carry Select Adder

O Carry Select Adder de 8 bits é construído reutilizando o subcircuito RCA já implementado. O primeiro bloco é um RCA de 4 bits com C_{in} fixo em zero. O segundo bloco é duplicado, um RCA com C_{in} zero e outro com C_{in} um, ambos recebendo os mesmos bits A e B e calculando simultaneamente. Dois multiplexadores fazem a seleção: um de 4 bits para os bits de soma e um de 1 bit para o carry de saída, ambos usando o C_{out} do primeiro bloco como sinal de seleção.

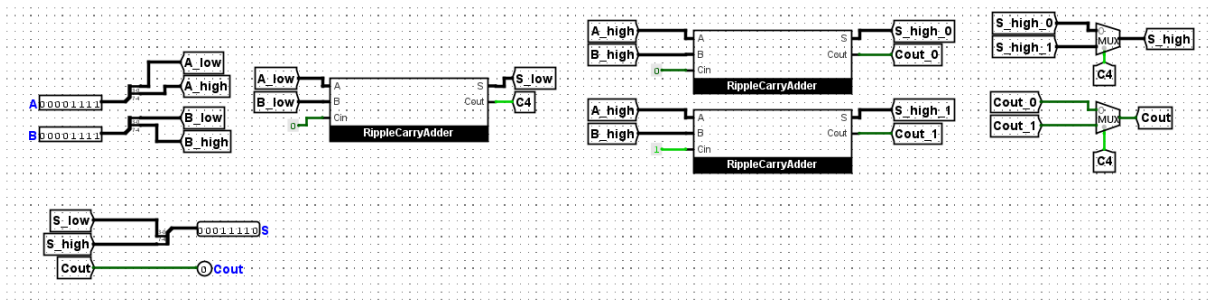


Imagem 5 - Circuito completo Carry Select Adder

3.5 Carry Save Adder

O CSA de 4 bits é implementado como uma camada de quatro Full Adders completamente independentes entre si — diferente de todos os somadores anteriores, não há nenhuma conexão de carry entre eles. Cada Full Adder recebe os bits A_i , B_i e C_i dos três operandos e produz um bit de soma e um carry de forma isolada. Os carries salvos são deslocados uma posição para a esquerda antes de entrar no somador final, representado pelo subcircuito RCA já implementado, que resolve os dois vetores e entrega o resultado completo.

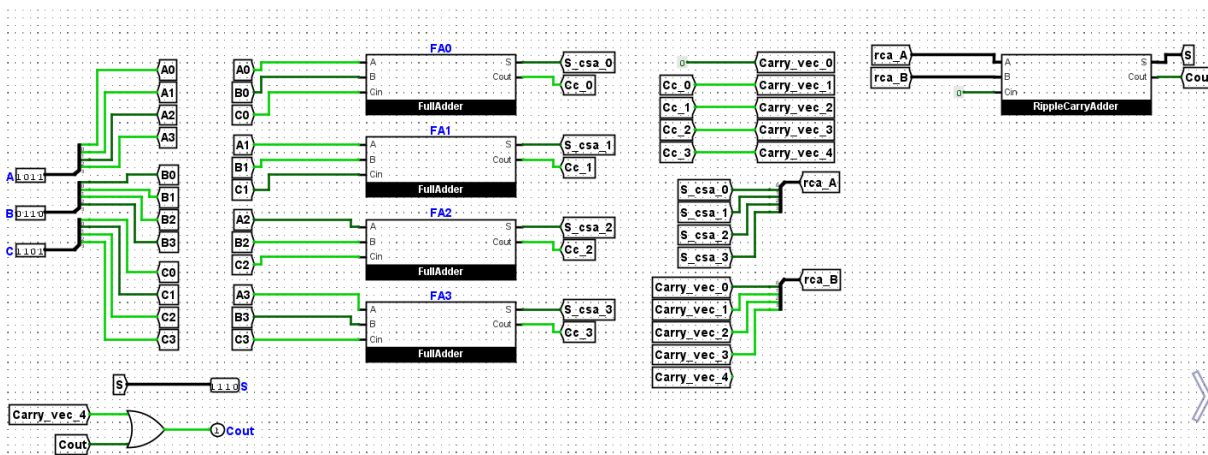


Imagem 6 - Circuito completo Carry Save Adder

3.7 Kogge-Stone Adder

O Kogge-Stone de 8 bits é o circuito mais complexo da série e é organizado em quatro faixas horizontais no canvas, cada uma correspondendo a uma etapa do cálculo. A primeira faixa contém a camada de G e P inicial, oito portas AND e oito portas XOR calculando os sinais de geração e propagação de cada bit em paralelo. As três faixas seguintes correspondem aos três níveis da árvore prefixo.

Cada nó da árvore é implementado como um subcircuito chamado prefix_operator, com quatro entradas (G alto, P alto, G baixo, P baixo) e duas saídas (G out, P out), implementando o operador prefixo com uma porta OR, duas portas AND e uma porta AND

de três entradas. A cada nível o passo dobra 1, 2 e 4 e os nós das posições sem par à esquerda passam seus valores diretamente para o próximo nível sem modificação.

Ao final do nível 3, cada posição tem seu par prefixo acumulado completo desde o bit 0. O G acumulado de cada posição i é diretamente o carry que entra na posição $i+1$. A camada final calcula os bits de soma com portas XOR, combinando o P inicial de cada bit com o carry correspondente.

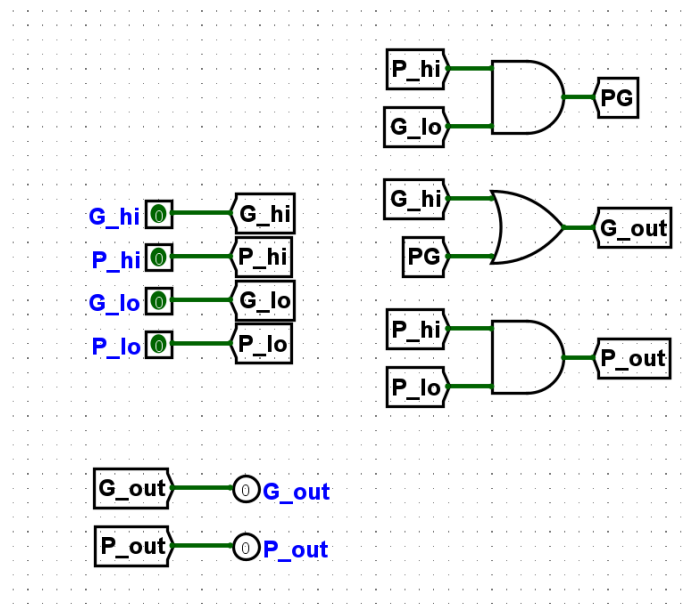


Imagem 7 - Circuito Prefix_operator

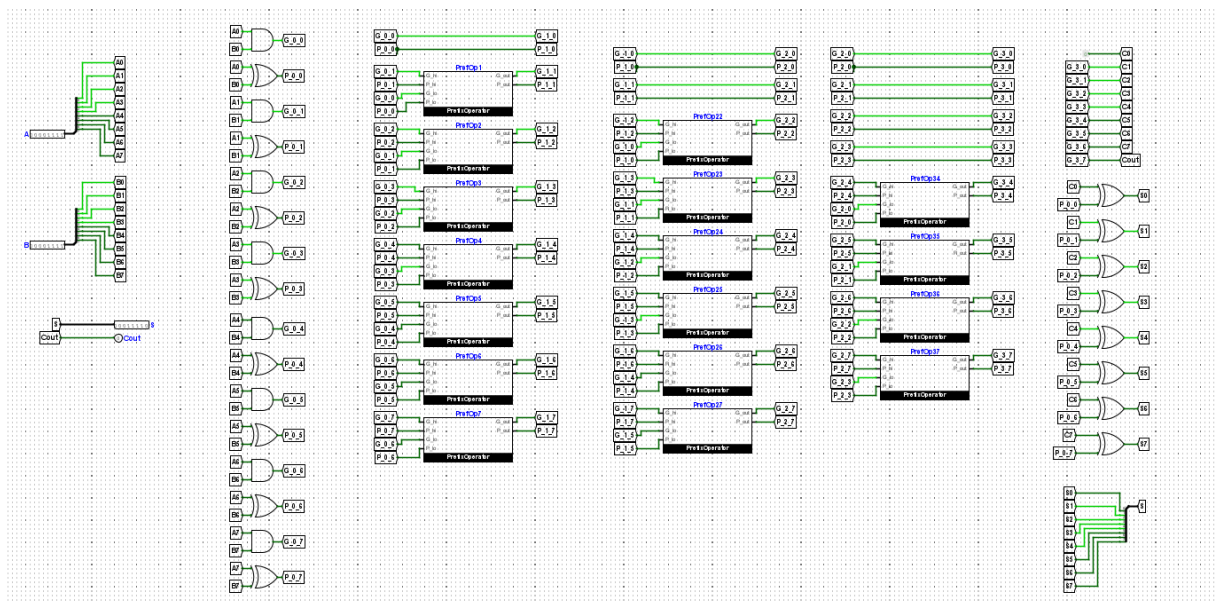


Imagem 8 - Circuito completo Kogge-Stone Adder

4. CONSIDERAÇÕES FINAIS

Este trabalho percorreu o desenvolvimento dos principais somadores digitais, partindo dos blocos mais elementares, o Half Adder e o Full Adder, até arquiteturas de alto desempenho como o Kogge-Stone Adder. Ao longo dessa trajetória, ficou evidente que cada arquitetura não surge de forma isolada, mas como resposta direta às limitações da anterior, formando uma progressão coerente guiada pelo mesmo problema central: a propagação do carry.

O Ripple Carry Adder estabelece a base conceitual do tema. Sua simplicidade de construção e regularidade estrutural o tornam o ponto de partida natural para o estudo de somadores, mas seu atraso linear revela com clareza o obstáculo que todas as arquiteturas seguintes buscam superar. O Carry Lookahead Adder representa a primeira resposta sistemática a esse obstáculo, antecipando os carries por meio dos sinais de geração e propagação. O Carry Select Adder adota uma abordagem diferente, eliminando a espera pelo carry por meio do paralelismo e da seleção, sem exigir a lógica de lookahead. O Carry Save Adder amplia o escopo do problema, mostrando que a soma de múltiplos operandos exige uma estratégia própria, e que postergar a resolução do carry pode ser uma vantagem. Por fim, o Kogge-Stone Adder generaliza os princípios do CLA em uma estrutura de árvore de prefixos, calculando todos os carries em tempo logarítmico e fundamentando as implementações encontradas em processadores modernos.

A implementação dos circuitos no Logisim Evolution permitiu verificar na prática os princípios discutidos na revisão teórica. A construção progressiva dos somadores cada um reutilizando subcircuitos dos anteriores reforça a natureza modular do design digital e demonstra que conceitos abstratos como geração e propagação de carry se traduzem diretamente em estruturas de portas lógicas observáveis e simuláveis.

Como desdobramentos futuros, seria relevante expandir a análise para somadores não abordados neste trabalho, como o Brent-Kung e o Han-Carlson, aprofundando a família dos somadores paralelos prefixo. Por fim, a análise de desempenho comparando os resultados de síntese com as previsões teóricas de atraso e área, representaria uma contribuição concreta à validação experimental das arquiteturas estudadas.

5. REFERÊNCIAS

KOGGE, Peter M.; STONE, Harold S. A parallel algorithm for the efficient solution of a general class of recurrence equations. **IEEE Transactions on Computers**, v. C-22, n. 8, p. 786–793, ago. 1973. Disponível em: <https://ieeexplore.ieee.org/document/5009159>. Acesso em: 4 mai. 2026.

HENNESSY, John L.; PATTERSON, David A. A new golden age for computer architecture. **Communications of the ACM**, v. 62, n. 2, p. 48–60, fev. 2019. Disponível em: <https://cacm.acm.org/research/a-new-golden-age-for-computer-architecture/>. Acesso em: 4 mai. 2026.

LOGISIM-EVOLUTION. **Repositório oficial**. Disponível em: <https://github.com/logisim-evolution/logisim-evolution>. Acesso em: 4 mai. 2026.

NESO ACADEMY. **Carry Lookahead Adder (Part 1) | CLA Generator**. YouTube, 28 out. 2014. Disponível em: <https://www.youtube.com/watch?v=PdHHCFGHdgQ>. Acesso em: 4 mai. 2026.

NESO ACADEMY. **Carry Lookahead Adder (Part 2) | CLA Adder**. YouTube, 28 out. 2014. Disponível em: <https://www.youtube.com/watch?v=9lyqSVKbyz8>. Acesso em: 4 mai. 2026.

NESO ACADEMY. **Full Adder**. YouTube, 20 out. 2014. Disponível em: <https://www.youtube.com/watch?v=RK3P9L2ZXk4>. Acesso em: 4 mai. 2026.

NESO ACADEMY. **Half Adder**. YouTube, 18 out. 2014. Disponível em: <https://www.youtube.com/watch?v=aLUY-s7LSns>. Acesso em: 4 mai. 2026.

PATTERSON, David A.; HENNESSY, John L. **Organização e projeto de computadores: a interface hardware/software**. 5. ed. Rio de Janeiro: Elsevier, 2014.